Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ЭССЕ

Дисциплина МДК 03.02

Модели качества программного обеспечения и целесообразность модулей адаптации

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3

1 МОДЕЛИ КАЧЕСТВА В DEVSECOPS.....4

2 АДАПТАЦИЯ LEGACY СИСТЕМЫ К СЕРВИСУ NET 8.....5

ЗАКЛЮЧЕНИЕ.....6

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....7

ВВЕДЕНИЕ

При интеграции старой системы с новым сервисом я считаю главным вопросом сохранение проверяемых свойств. Успешная сборка .NET 8 не доказывает правильность преобразования данных, а перечень стандартов без критериев не помогает решить, можно ли выпускать изменение. Я рассматриваю SQuaRE как язык требований, модуль адаптации — как границу между несовместимыми моделями [2].

В электронной библиотеке старый каталог может хранить путь и статус одной строкой, а новый API разделяет идентификатор, права и карантин. Я связываю выбор архитектуры с качеством результата, стоимостью сопровождения и свидетельствами, которые остаются после автоматического конвейера.

1 МОДЕЛИ КАЧЕСТВА В DEVSECOPS

Модель качества полезна до написания тестов. Защищённость становится требованием «Reader не загружает книгу», надёжность — требованием не оставлять часть архива после отказа. Восьмихарактеристическая национальная модель удобна для учебного профиля; актуальная ISO/IEC 25010:2023 с девятью характеристиками должна учитываться отдельно [1] [3].

Я не считаю средний балл достаточным основанием для выпуска. Высокая сопровождаемость не компенсирует обход прав, а быстрый ответ — повреждение данных. Поэтому нужны блокирующие критерии. Тест опасного пути должен пройти независимо от общей оценки профиля. Профиль показывает приоритеты, решение принимается по последствиям отказа.

В DevSecOps проверки распределяются по жизненному циклу: до реализации — угрозы и критерии; при изменении — анализ и тесты; перед выпуском — результаты и зависимости; после выпуска — инциденты. Это согласуется с рекомендациями SSDF о включении безопасной разработки в существующий процесс. Автоматический отчёт остаётся свидетельством проверки, а не гарантией отсутствия неизвестных ошибок [5].

Для небольшого проекта я выбрал бы короткий набор обязательных барьеров: сборку, тесты прав и форматов, отсутствие диагностических утечек. Дополнительная проверка оправдана, когда снижает известный риск. Ложные срабатывания нужно разбирать и исправлять, иначе существенные находки потеряются среди привычных предупреждений.

2 АДАПТАЦИЯ LEGACY СИСТЕМЫ К СЕРВИСУ NET 8

Адаптер нужен при различии смысла данных, а не только транспорта. В старой библиотеке «доступна» может означать наличие файла на диске; в новом API — разрешённую публикацию. Прямое копирование флага способно открыть непроверенную книгу читателю. Я бы преобразовывал старый статус в состояние импорта, разрешая публикацию после предусмотренного контроля.

Anti-corruption Layer изолирует новую модель от особенностей старой системы. В него выносятся преобразования полей, кодировок, идентификаторов и ошибок. Тогда доменная модель .NET 8 не обязана повторять случайные решения legacy-базы. Вместе с пользой появляются задержка и стоимость сопровождения; Microsoft прямо рассматривает эти компромиссы в описании шаблона [4].

Я разделил бы контракт адаптера и доступ к источнику. Новый сервис получает устойчивый DTO; неизвестный статус или неправильный идентификатор отклоняется. Повторная доставка требует идемпотентности: одна старая запись не должна создавать несколько книг. Ошибки преобразования нужно хранить отдельно, чтобы оператор мог безопасно повторить импорт. Эти свойства закрепляются тестами и не возникают автоматически от наличия слоя.

Отдельный микросервис оправдан не всегда. Если модели совпадают, обмен разовый и источник контролируется, достаточно внутреннего адаптера. Если legacy меняется независимо и содержит опасные допущения, изоляция снижает связанность и позволяет заменять её постепенно. Контракт проверяется на корректной записи, отсутствующем поле, неизвестном статусе, повторном идентификаторе и отказе источника. После вывода старой системы адаптер следует удалить или упростить.

ЗАКЛЮЧЕНИЕ

Я рассматриваю SQuaRE как основу проверяемых требований, DevSecOps — как способ регулярно получать свидетельства их выполнения. Адаптер оправдан, когда защищает новую модель от чужой семантики и локализует изменения. Для Library System это различение наличия файла и разрешённой публикации. Переход на .NET 8 сам по себе не устраняет ошибки данных и доступа; архитектурное решение зависит от риска и цены поддержки.

Материалы работы размещены в репозитории: https://github.com/rob4ik72-del/MDK0302-LibrarySystem.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р ИСО/МЭК 25010-2015. Системная и программная инженерия. Модели качества систем и программных продуктов. — URL: https://protect.gost.ru/gost/details/e5744d86-5a17-4909-96da-b38691326f0c (дата обращения: 08.11.2026) [1].

2. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [2].

3. ISO/IEC 25010:2023. Systems and software engineering. SQuaRE. Product quality model. — URL: https://www.iso.org/standard/78176.html (дата обращения: 08.11.2026) [3].

4. Microsoft Azure Architecture Center. Anti-corruption Layer pattern. — URL: https://learn.microsoft.com/en-us/azure/architecture/patterns/anti-corruption-layer (дата обращения: 08.11.2026) [4].

5. NIST SP 800-218. Secure Software Development Framework Version 1.1. 2022. — URL: https://csrc.nist.gov/pubs/sp/800/218/final (дата обращения: 08.11.2026) [5].